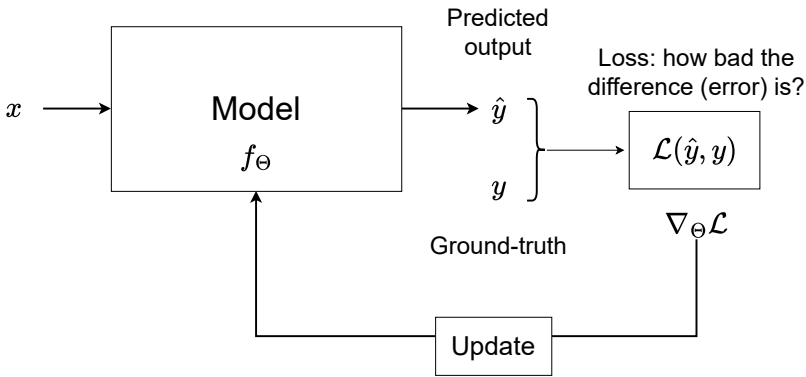
DEEP LEARNING

Fundamentals of Deep Learning: Part III

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap: A Modular Learning Framework



Recap: Two types of Problems

- Regression is a problem in which given an input $\mathbf{x}$, the parameterized model f_Θ outputs a scalar value:

$$\hat{y} = f_\Theta(\mathbf{x}) = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d + \theta_0.$$

- Classification is a problem in which given an input $\mathbf{x}$, the parameterized model f_Θ (also called a classifier) outputs a probability distribution:

$$\hat{\mathbf{y}} = f_\Theta(\mathbf{x}) = \texttt{softmax}(z_\Theta(\mathbf{x})),$$

  where the final class label is obtained with an arg max operation:

$$c^* = \arg\max_{c \in \mathcal{C}} \hat{\mathbf{y}}$$

- For example, $\hat{\mathbf{y}} = [0.1, 0.1, \mathbf{0.6}, 0.05, 0.15]$, the predicted class label is $c^* = 2$ (the element at index 2 is the highest value of the vector).

Today

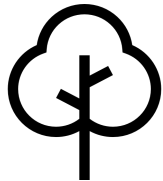
- The problem of generalization
- Polynomial regression
- Underfitting and overfitting
- Bias-Variance tradeoff
- Regularization
- Validation techniques

Table of Contents

Generalization Problem

Generalization Problem

Underlying concept



Different kinds of trees

Generalization Problem

- The primary goal of any deep learning algorithm is not just to do well during the training phase, but to do well during testing.
- Thus, the problem is to discover *patterns* during training that generalize well to many test inputs.
- As a practical example, once we have trained a classifier for classifying diseased from healthy individuals, we want the classifier to work well on the data of new patients or a classifier that generalizes well.
- The problem of generalization is one of the most important considerations while designing, and training a DL model.
- Failure to generalize can stem from two reasons:
  - Underfitting
  - Overfitting

Polynomial Regression

Polynomial Regression

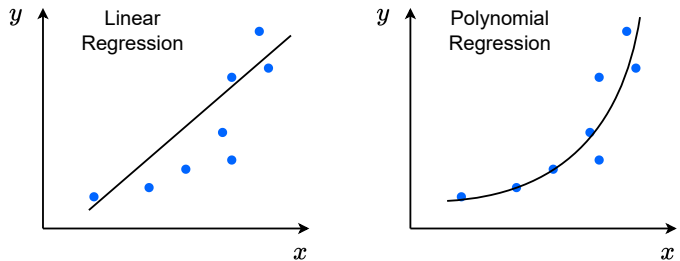
- We will study the generalization problem through the use of polynomial regression.
- Polynomial regression is similar to linear regression, except the hypothesis space is polynomial functions rather than linear functions, ie,

$$y = f_\Theta(x) = \sum_{k=0}^{K} \theta_k x^k,$$

  where K is the degree of the polynomial.
- If we set $K = 1$, we get the familiar linear regression equation, making it a special case of polynomial regression.

Polynomial Regression

- Why polynomial regression? The relationship between the input x and the output y may *not* always be linear.



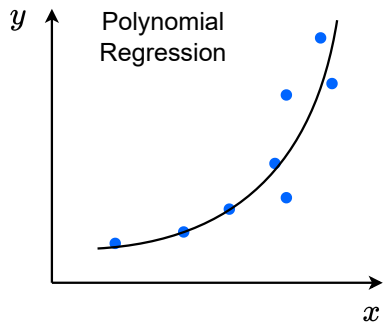
- In cases, where the input x has a non-linear relationship with the output y, then we use polynomial regression.
- Question: which polynomial function (or what value of K) is suitable for the above example?

Polynomial Regression

- Setting $K = 2$, we get the polynomial function shown on the right figure:

$$y = f_\Theta(x) = \sum_{k=0}^{2} \theta_k x^k,$$
$$= \theta_0 + \theta_1 x + \theta_2 x^2$$

- Important: The above equation is non-linear in x because x has powers more than 1, but still linear in θs because the power of each θ is still equal to 1.



Polynomial Regression

- Polynomial repression can be *transformed into* linear regression problem using a neat trick. After that, we can reuse all the things we have learned previously for least-squares linear regression.
- The trick is *transform* the input x by applying a feature transformation $\phi(x)$ on x as:

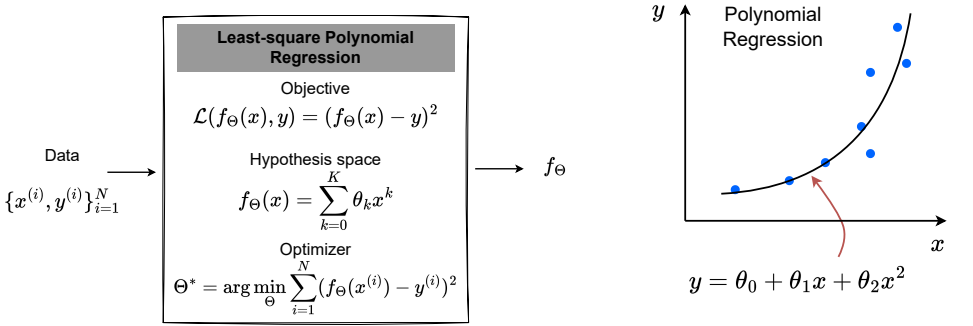
$$x \overset{\phi}{\mapsto} \underbrace{\begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}}_{\text{new input}}$$

- Following the dot product formulation of linear regression:

$$y = \sum_{k=0}^{2} \theta_k x^k = \Theta^\mathsf{T} \phi(x),$$

where $\Theta = [\theta_0, \theta_1, \theta_2]$ are the parameters of the linear regression model.

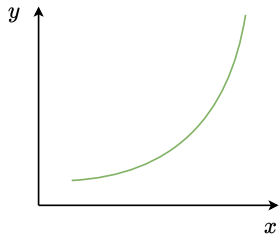
Polynomial Regression



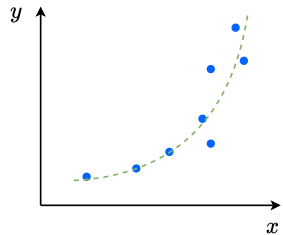
- We can apply all the previously learned concepts to find the unknown parameters, except this time we call it least-square polynomial regression.

Back to the Generalization Problem

Polynomial Regression as a Lens into Generalization



The true function

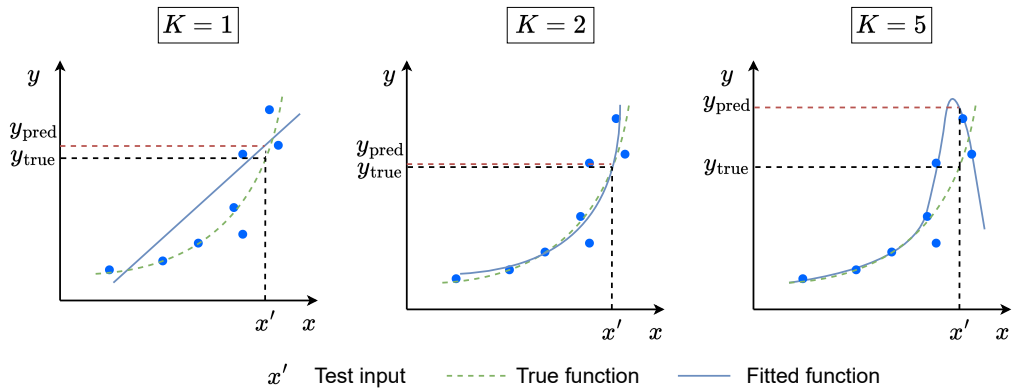


What we have access to: only the (blue) training data points

- The training data points do not fall exactly on the true function (or green line) because of observation noise.
- Goal: given a set of training data points, we need to find the right model, or in other words, the right value of K.
- Challenge: The value of K in polynomial regression influences how good is a model, or how good is it at generalization.

Influence of K on the Hypothesis Space

- If we vary the value of K we will have different kinds of models (or fits), as shown below.



- Question: Which value of K will you settle for?

Overfitting versus Underfitting

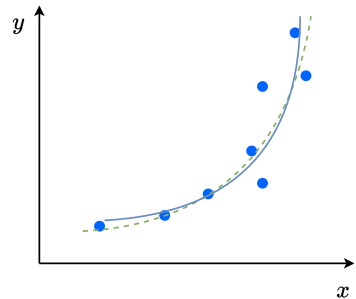
- Some observations:
  1. As K increases (ie, $K = 5$), the function expands to include ever more training data points (blue points), and results in a wiggly or curvy fit.
  2. As K decreases (ie, $K = 1$), the function becomes too simple and can hardly fit the training data points.
  3. Only for the right value of K (ie, $K = 2$), the function can learn the true underlying pattern.
- Case (1) is called overfitting, where the fit on training points is great, but can give poor prediction on a test sample $\approx$ memorizing before an exam
- Case (2) is called underfitting, where the fit on training points is poor, and so is the prediction on a test sample $\approx$ studying poorly before an exam
- Case (3) is the *right fit* as it fits the training data points reasonably well and gives good prediction on a test sample $\approx$ studying well all the underlying concepts

Kinds of Errors: Approximation Error

- Approximation error is the gap between the fitted function (blue line) and the training data points (blue points).
- Given a training dataset $\{x_{\text{train}}^{(i)}, y_{\text{train}}^{(i)}\}_{i=1}^{N}$, the approximation error is defined as:

$$J_{\text{approx}} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_{\Theta}(x_{\text{train}}^{(i)}), y_{\text{train}}^{(i)})$$

- In some books, minimizing J_{approx} is called Empirical Risk Minimization.



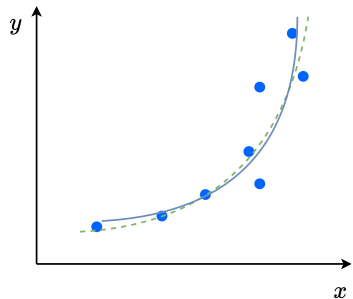
Kinds of Errors: Generalization Error

- Generalization error is the gap between the fitted function (blue line) and the true function (dotted green line).
- In theory, generalization error is the expected cost (or loss) we would incur if we sampled a new test point at random,ie:

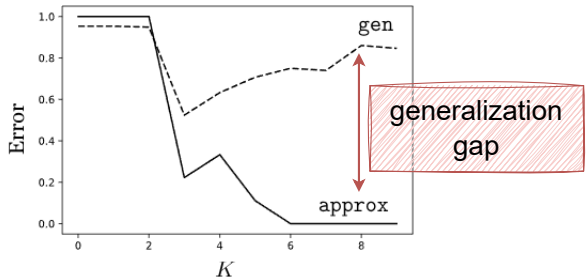
$$J_{\text{gen}} = \mathbb{E}_{(x,y)\sim p_{\text{data}}} \mathcal{L}(f_\Theta(x), y)$$

- In practice, generalization error is *approximated* on a held out validation set: $\{x^{(i)}_{\text{val}}, y^{(i)}_{\text{val}}\}_{i=1}^{N}$:

$$J_{\text{gen}} \approx \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\Theta(x^{(i)}_{\text{val}}), y^{(i)}_{\text{val}})$$



Relationship between Approximation and Generalization Error



- Typically, approximation error descreases with increasing K, but we really care about the generalization error.
- As K increases, the generalization error (or the generalization gap) increases.
- This phenomenon is also prevalent in Deep Neural Networks. As the neural networks becomes bigger and bigger, the approximation error goes down and the generalization error increases.

Bias-Variance Tradeoff

Bias-Variance Tradeoff

- Generalization error measures how well your model generalizes to data it hasn't seen before.
- The generalization error can be decomposed into three parts (proof omitted):

$$\text{expected error} = (\text{bias})^2 + \text{variance} + \text{noise}$$

- There is a trade-off between the variance and bias terms in the error, and hence the name bias-variance tradeoff.
- Note, a good understanding of bias-variance trade-off allows us to debug and create good models that generalize well.

Bias-Variance Tradeoff

- We will first introduce some terms.
- Let's assume we are in the linear regression setting, where we are predicting the number of people at the beach $Y = y$ given some features $X = \mathbf{x}$ (e.g., temperature outside, day of the week, and so on), where X and Y are random variables (r.v).
- We could imagine two different days in a month, with exactly the same features, but with different number of people at the beach. Thus, there is a **distribution over possible labels**. The expected label (given an $\mathbf{x}$) is given as:

$$\bar{y} = \mathbb{E}_{y|\mathbf{x}}[Y]$$

- The expected label denotes the label you would expect to obtain, given a feature vector $\mathbf{x}$.
- In simple words, expectation $=$ the average outcome if we could try (or observe) something many many times.

Bias-Variance Tradeoff

- Let's say that we draw our training data $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ from a distribution p_{data}.
- We use a linear regression algorithm to train a model on the dataset $\mathcal{D}$. We denote this as $f_{\mathcal{D}}$.
- After training the model $f_{\mathcal{D}}$, we are interested in finding what is the error on new data points. The expected test error (given $f_{\mathcal{D}}$) is denoted as:

$$\mathbb{E}_{(\mathbf{x},y)\sim p_{\text{data}}}[(f_{\mathcal{D}}(\mathbf{x}) - y)^2]$$

- In other words, we sample many data points from the distribution p_{data} and average the errors to get the expected test error for the specific model that we trained on $\mathcal{D}$.

Bias-Variance Tradeoff

- We can collect not just one dataset $\mathcal{D}$ of N data points, but many of them from the distribution P^N. Thus $\mathcal{D}$ is a r.v.
- Since $f_{\mathcal{D}}$ is a function of a r.v. $\mathcal{D}$, we can say that there is a **distribution of models** $f_{\mathcal{D}}$. Thus, we can compute the expected classifier as:

$$\bar{f} = \mathbb{E}_{\mathcal{D} \sim P^N}[f_{\mathcal{D}}]$$

- An intuitive way of thinking about $\bar{f}$ is that: if we had infinitely many training sets, and train a classifier $f_{\mathcal{D}}$ on each training set; and average the prediction for a given test sample $\mathbf{x}$, its value will be equal to $\bar{f}(\mathbf{x})$.
- A simpler name for the expected classifier is "best possible classifier".

Bias-Variance Tradeoff

- We can write the expected test error or generalization error as:

$$\mathbb{E}_{\substack{(\mathbf{x},y)\sim p_{\text{data}} \\ \mathcal{D}\sim P^N}}[(f_{\mathcal{D}}(\mathbf{x}) - y)^2]$$

- The term above says:
  1. Sample a training dataset $\mathcal{D}$ containing N data points from a data distribution P^N.
  2. Train a classifier $f_{\mathcal{D}}$ on the dataset $\mathcal{D}$.
  3. Sample a test data point $(\mathbf{x}, y)$ from p_{data} and compute the error.
  4. Repeat the above steps infinitely many times.

Bias-Variance Tradeoff

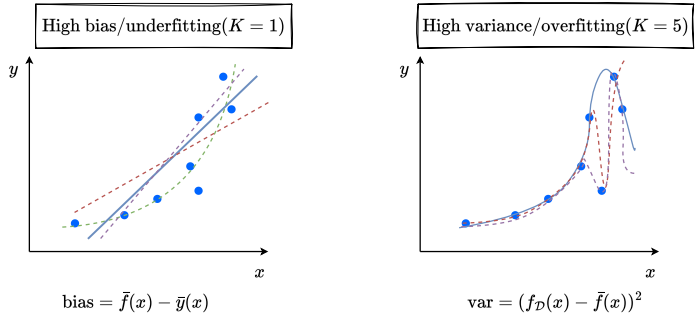
- We can decompose the expected test error into three interpretable terms as follows:

$$\underbrace{\mathbb{E}_{\mathbf{x},t,\mathcal{D}}[(f_{\mathcal{D}}(\mathbf{x}) - t)^2]}_{\text{Expected test error}} = \underbrace{\mathbb{E}_{\mathbf{x},\mathcal{D}}[(\overbrace{f_{\mathcal{D}}(\mathbf{x})}^{\text{a classifier}} - \overbrace{\bar{f}(\mathbf{x})}^{\text{best classifier}})^2]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathbf{x},y}[(\overbrace{\bar{y}(\mathbf{x})}^{\text{expected label}} - \overbrace{y}^{\text{actual label}})^2]}_{\text{Noise}}$$

$$+ \underbrace{\mathbb{E}_{\mathbf{x}}[(\overbrace{\bar{f}(\mathbf{x})}^{\text{best classifier}} - \overbrace{\bar{y}(\mathbf{x})}^{\text{expected label}})^2]}_{\text{Bias}^2}$$

- **Variance**: This term tells how much your classifier varies if you train on a different training set. If we have the best possible classifier, how far off are we.
- **Bias**: What is the inherent error that you obtain from your classifier even with infinite training data? How "biased" is my model towards a particular solution (e.g., linear models).
- **Noise**: By how much is the collected label value different from the expected label.

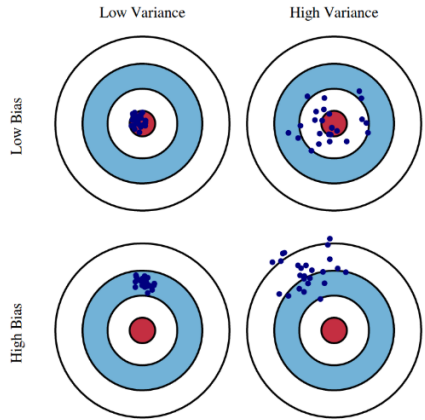
Bias-Variance Tradeoff



- If the original data is non-linear (ie, follows a parabolic curve) and if we train a simple linear model ($K = 1$), then no matter the amount of training data we have, the model will always be biased towards a linear solution.
- If we train a more "complex model" ($K = 5$), then a model will fit each training dataset really well, but every model will produce a different (or varying) prediction due to overfitting on the noise.

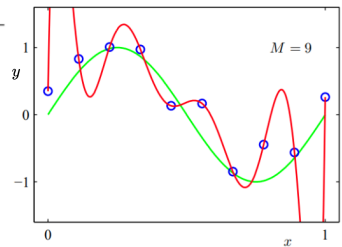
Bias-Variance Tradeoff

- Consider the analogy of model's predictions as playing the game of darts. 🎯
- In Low Bias (LB) and Low Variance (LV), the model's predictions are always dead on the center.
- In LB and High Variance (HV), the predictions vary a lot, but in expectation they will hit the center.
- In High Bias (HB) and LV, they are off the center (i.e., biased), but don't vary a lot.
- In HB and HV, not only they are off from the expected label (i.e., center) but also varies a lot.



Model Complexity

	$K=0$	$K=1$	$K=3$	$K=9$
θ_0	0.19	0.82	0.31	0.35
θ_1		-1.27	7.99	232.37
θ_2			-25.43	-5321.83
θ_3			17.37	48568.31
θ_4				-231639.30
θ_5				640042.26
θ_6				-1061800.52
θ_7				1042400.18
θ_8				-557682.99
θ_9				125201.43



$$y = \theta_0 + \theta_1 x + \cdots + \theta_9 x^9$$

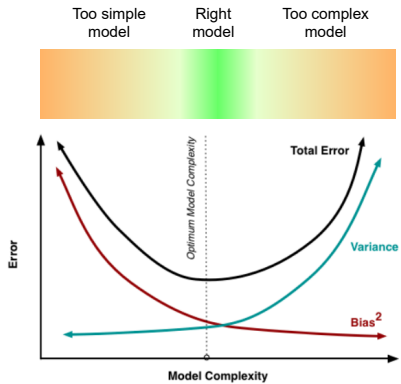
- As K increases, the magnitudes of the coefficients (ie, weights θs) get larger (or the model becomes more complex).
- One way to interpret the effect of large weights is: small change in the value of x will lead to large changes in the model's prediction.

Bias-Variance Tradeoff and Model Complexity

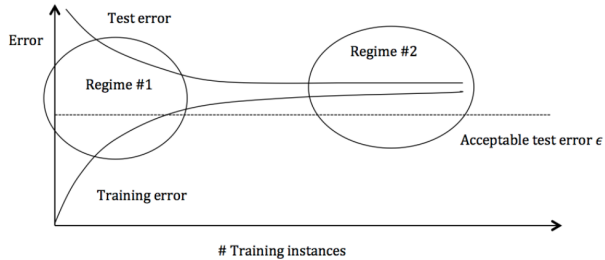
- Recall that the expected error is a sum of three terms:

$$\text{expected error} = (\text{bias})^2 + \text{variance} + \text{noise}$$

- When the model is too simple (e.g., using a linear model for data that is non-linear) the error will be dominated by the bias term.
- When the model is too complex (e.g., a model with way too many parameters) the error will be dominated by the variance term.
- When the model complexity is just about right, we get the lowest generalization error.



How to Detect High Bias and High Variance?



Regime	What to look for?	How to overcome?
Regime 1 (a.k.a Overfitting)	- High gap between train and test error - Training error lower than ϵ - Test error higher than ϵ	- Regularization - Get more training data - Bagging
Regime 2 (a.k.a Underfitting)	- Training error is higher than ϵ - So is the test error	- Use more complex models - Add features

Regularization

Regularization

- Regularization refers to mechanism that penalize function complexity so that we avoid learning a function that is too flexible or a function that overfits.
- We add a term to the objective (or loss) function, and call the final objective function as the regularized objective function:

$$J(\Theta) = \underbrace{\frac{1}{N}\mathcal{L}(f_\Theta(\mathbf{x}^{(i)}), y)}_{\text{original objective function}} + \underbrace{\lambda R(\Theta)}_{\text{regularizer}},$$

  where λ is a hyperparameter that controls the strength of the regularization.
- Intuitively, the regularizer's role is to reduce the magnitude of the weights (ie, θs), and enforce a simpler solution.

ℓ_2 Regularization

- One of the ways to make the weights of a model small is by penalizing the ℓ_2 norm of the parameters of the model:

$$R(\Theta) = \|\Theta\|_2 = \sqrt{\theta_0^2 + \theta_1^2 + \cdots + \theta_K^2}$$

- Penalizing the ℓ_2 norm means making the above term as small as possible. Note that it does not depend on the data, but only on the parameters of the model.
- In practice, we minimize the square of the ℓ_2 norm:

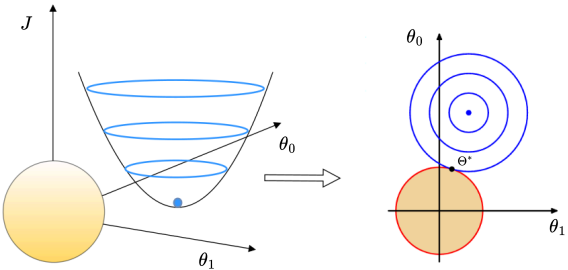
$$R(\Theta) = \frac{1}{2}\|\Theta\|_2^2 = \frac{1}{2}\sum_{k=0}^{K} \theta_k^2.$$

ℓ_2 Regularization: Geometric Interpretation

- Consider a model with parameters $\Theta = [\theta_0, \theta_1]$. The regularized objective function is:

$$J(\theta_0, \theta_1) = \frac{1}{N}\mathcal{L}(f_\Theta(\mathbf{x}^{(i)}), y) + \frac{1}{2}(\theta_0^2 + \theta_1^2).$$

- The regularization term is nothing but a circle.



- The solution would be the point where the two curves intersect. This means, although we could not reach the lowest point in the blue curve, we have a simpler solution where the magnitudes of $[\theta_0, \theta_1]$ are smaller.

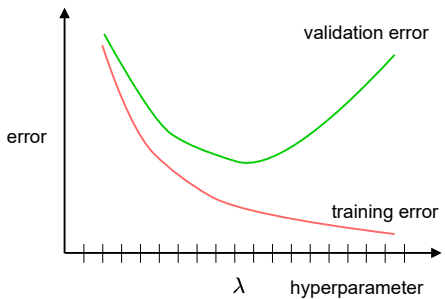
Validation Techniques

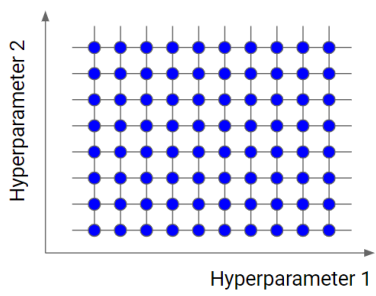
Cross Validation

- Question: How do we decide on a good model (ie, value of K in regression) and choose the hyperparameters (ie, λ in regularization)?
- Cross validation is a method to evaluate the performance of a model by dividing the dataset into two parts:
  - Training set – used to train the model.
  - Testing set – used to test how well the model generalizes to unseen data.
- To avoid overfitting and get a realistic estimate of the model's performance on new, unseen data.
- Another common strategy is to split the dataset into three parts:
  - Training set – to train the model.
  - Validation set – to find optimal hyperparameters.
  - Testing set – to check how well the model generalizes to unseen data.
- Common splits: 80%/20% or 60%/20%/20%.

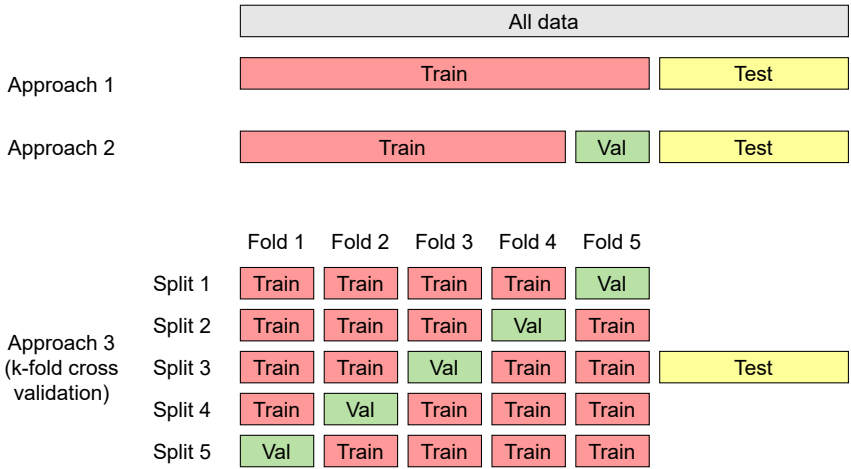
Grid Search

- Grid Search is a method for hyperparameter tuning – it exhaustively tries all combinations of specified hyperparameter values to find the best-performing setup for your model.
- The model is trained and evaluated on each combination using cross-validation.
- The best hyperparameter(s) is/are selected.





K-fold Cross Validation

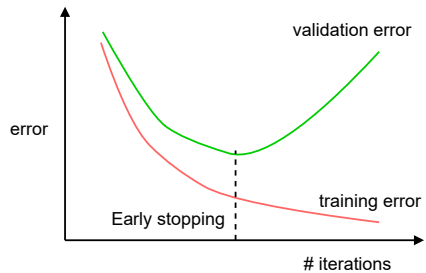


K-fold Cross Validation

- K-fold cross validation works as follows:
  - The training set is split into k equal parts or folds.
  - For each split:
    - Train a model using $k-1$ of the folds as training data.
    - The resulting model is validated on the remaining part of the data.
  - The performance measure reported by k-fold cross-validation is then the average of the values computed for each split.
- Advantages:
  - Efficient use of data, especially when the size of the dataset is small.
  - Reduces the chance of misleading results due to an unlucky split.
- K-fold cross-validation technique is often used for searching for the best hyperparameters – the hyperparameters that produce the best average performance.
- When $k = N$, N being the size of the training data, we get leave-one-out (LOO) cross-validation.

Early Stopping

- Early stopping is a regularization technique used to prevent overfitting during training by stopping the training process early when performance on a validation set starts to get worse.
- How does it work?
  - Split your data into train and validation sets.
  - Track validation loss at every (or a pre-defined number of) iterations.
  - Stop training if validation loss doesn't improve.



Outro

- The problem of generalization is relevant in all kinds of deep learning models.
- A DL model that achieves 100% accuracy on the training set and very poor accuracy on unseen data is not useful.
- As a DL engineer your goal is to build models that generalize well to unseen data.
- Today we learned how to probe for generalization and what to do when a model overfits (more common) and underfits.
- In future lectures, we will explore more techniques to improve generalization.

Reading materials

- Dive into Deep Learning (`https://d2l.ai/index.html`). Chapters 3.6
- Deep Learning (`https://www.deeplearningbook.org/`). Chapters 5 (5.1, 5.2, 5.3), 7.1 (optional)

Credits and Acknowledgements

- Some slides have been adapted from
  - Torralba, Isola and Freeman
  - F. Amir
  - W. Kilian